
Gradient Boosting in a location scale regression framework

Term Paper

by

Meyer, Rudi Murat
rudi.meyer@stud.uni-goettingen.de

Vila, Ferran
ferran.vila@stud.uni-goettingen.de

Wachtel, Sophie Eileen
sophie.wachtel@stud.uni-goettingen.de

Wais, Ben Lenard
benlenard.wais@stud.uni-goettingen.de

Module M.WIWI-QMW.0011
(Advanced Statistical Programming with R)

September 15, 2026

Contents

List of Figures	II
List of Tables	III
Listings	IV
1 Introduction	1
2 Methods	1
2.1 The Gaussian-location-scale model	1
2.2 Boosting the Gaussian-location-scale model	2
3 Usage and implementation	5
3.1 Usage	5
3.2 Implementation	6
4 Simulation study	8
4.1 Simulation Study Design	8
4.2 Simulation Study Results	13
5 Conclusions	16
Bibliography	V
A Appendix	VII
A.1 Additional Figures and Tables	VII
A.2 Reproducibility	XXIII
A.3 Package Manual	XXIV
Declaration	XXXVI

List of Figures

A1	Graphical output of the <code>plot()</code> method for a fitted <code>boost_lmls</code> model.	VIII
A2	Reduced two-dimensional grid for tuning $m_{\text{stop},\mu}$ and $m_{\text{stop},\sigma}$. .	IX
A3	Predictive performance of cyclic and non-cyclic boosting with linear base-learners.	XIX
A4	Variable-selection performance of cyclic and non-cyclic boosting with linear base-learners.	XX
A5	Performance of non-cyclic linear and spline base-learners in the nonlinear DGP 4.	XXI
A6	Distribution and diagnostics of the NLL in DGP 4.	XXII
A7	Code coverage report.	XXIII

List of Tables

1	Arguments and default values of <code>boost_lmls()</code>	5
2	Overview of the data-generating processes.	10
3	Overview of the factors varied across the simulation scenarios.	10
4	Number of parameter updates until convergence.	14
A1	Overview of the ADEMP framework.	X
A2	Recovery performance RMSE_μ , by DGP, n and ρ_X	XI
A3	Recovery performance $\text{RMSE}_{\eta_\sigma}$, by DGP, n and ρ_X	XII
A4	Recovery performance NLL, by DGP, n and ρ_X	XIII
A5	Variable-selection performance TPR μ , by DGP, n and ρ_X	XIV
A6	Variable-selection performance TPR σ , by DGP, n and ρ_X	XV
A7	Variable-selection performance FPR μ , by DGP, n and ρ_X	XVI
A8	Variable-selection performance FPR σ , by DGP, n and ρ_X	XVII
A9	Performance of linear and spline base-learners in DGP 4.	XVIII

Listings

1	Fitting a location-scale model with boost_lmls	5
A1	Example output of summary() for a boost_lmls object	VII
A2	Example of a prediction using the predict() method	VIII
A3	Tuning step length and stopping iterations with tune_lmls . .	IX

1 Introduction

In recent years generalized additive models for location, scale and shape (GAMLSS) have gained rising popularity and have been applied to a wide range of applications (Mayr et al., 2010). Introduced by Rigby & Stasinopoulos (2005) they are a flexible alternative to GLM and GAM approaches. Unlike those, GAMLSS make it possible to not only model the mean (location) but also further distribution parameters like the variance (scale) or shape, which can be particularly useful when dealing with, for example, heteroskedasticity.

Typically the estimation of GAMLSS relies on a combination of backfitting-type Gauss-Newton algorithms with AIC-based selection of variables (Thomas et al., 2018). However, this approach comes with limitations, as estimation becomes infeasible for high-dimensional data (Mayr et al., 2012), and variable selection tends to be “computationally almost impossible” for a large number of covariates (Mayr et al., 2010). By both providing built-in variable selection and accommodating high-dimensional data, gradient boosting can offer a solution to these problems.

Several R packages adapted different boosting procedures: While **XGBoost** (Chen & Guestrin, 2016) is a widely used implementation of tree-based gradient boosting, **mboost** (Hofner et al., 2012) implements component-wise gradient boosting. Building on the latter, **gamboostLSS** (Hofner et al., 2016) extends boosting to the GAMLSS framework. In this work, we introduce **boost_lmls**, an R package implementing component-wise gradient boosting for a special case of GAMLSS, the Gaussian Location-Scale Model (GLSM).

The paper is structured as follows: In Section 2 we give a theoretical introduction to both model and gradient boosting algorithms. Next the usage of the **boost_lmls**-package is demonstrated and its implementation clarified (Section 3). Finally, the functionality of the package is demonstrated and its different methods are compared empirically in a simulation study in Section 4.

2 Methods

2.1 The Gaussian-location-scale model

The Gaussian-location-scale model is a special case of the GAMLSS framework, where not only the mean μ is modelled as a function of its covariates, but also the standard deviation σ . For independent observations Y_i , the model can be written as

$$Y_i \stackrel{\text{ind.}}{\sim} \mathcal{N}(f_\mu(x_i), (\exp(f_\sigma(z_i)))^2).$$

where $f_\mu(x_i)$ and $f_\sigma(z_i)$ respectively denote the predictor functions for the location and scale parameters. While it is possible, the vectors x_i and z_i are not necessarily required to contain the same covariates, allowing different sets of explanatory variables to influence the conditional mean and scale.

To relate the general formulation to the familiar linear regression framework, we briefly consider the linear special case. In this case, the predictor functions are given by linear combinations of the covariates, resulting in

$$Y_i \stackrel{\text{ind.}}{\sim} \mathcal{N}\left(x_i^\top \beta, (\exp(z_i^\top \gamma))^2\right),$$

Here, β and γ determine the effects of the covariates on the conditional location and scale, respectively.

2.2 Boosting the Gaussian-location-scale model

Since the package targets GLSMs only, the following methods are restricted to two distribution parameters, μ and σ , under the assumption that the response variable follows a normal distribution. For a more general introduction to gradient boosting, see Hofner et al. (2012); for its extension to GAMLSS, see Hofner et al. (2016) and Thomas et al. (2018).

Component-wise gradient boosting is an iterative fitting method: In each iteration the algorithm aims to minimize a loss function. Instead of updating all predictors at once, only a subset will be adjusted. For that, candidate *base learners* are fitted separately to the negative gradient of the loss function (Bühlmann & Hothorn, 2007).

In case of the GLSM the loss function is the negative log-likelihood of the Gaussian distribution (Zhang et al., 2021). For the location parameter, the negative gradient is therefore

$$\frac{y^{(i)} - \hat{\mu}(x^{(i)})}{\hat{\sigma}(x^{(i)})^2}$$

and for the scale parameter, using the log link, the corresponding negative gradient is

$$\frac{(y^{(i)} - \hat{\mu}(x^{(i)}))^2}{\hat{\sigma}(x^{(i)})^2} - 1.$$

Only the best performing base learners will be selected, scaled by step size ν and then added to the predictor, which will gradually build up. The maximum number of boosting iterations is controlled by a stopping parameter, denoted by `mstop`. Depending on the boosting scheme, this can either be a

Algorithm 1: Cyclic component-wise gradient boosting for GLSMs.
Adapted from Hofner et al. (2016).

Initialization

1. Initialize $\hat{\eta}_\mu^{[0]}$ (identity link) and $\hat{\eta}_\sigma^{[0]}$ (log link) with offset values; set $\hat{\mu}^{[0]} = \hat{\eta}_\mu^{[0]}$ and $\hat{\sigma}^{[0]} = \exp(\hat{\eta}_\sigma^{[0]})$.
2. Specify base learners $h_{\mu 1}(\cdot), \dots, h_{\mu J_\mu}(\cdot)$ and $h_{\sigma 1}(\cdot), \dots, h_{\sigma J_\sigma}(\cdot)$.

For $m = 1$ **to** $\max(m_{\text{stop}, \mu}, m_{\text{stop}, \sigma})$:

3. **Update** μ : If $m \leq m_{\text{stop}, \mu}$:

- (a) Compute $u_\mu^{(i)} = \frac{y^{(i)} - \hat{\mu}^{[m-1]}(x^{(i)})}{\hat{\sigma}^{[m-1]}(x^{(i)})^2}$ for $i = 1, \dots, n$.

- (b) Fit each base learner $h_{\mu j}(\cdot)$ to u_μ ; select

$$j_\mu^* = \operatorname{argmin}_j \sum_{i=1}^n \left(u_\mu^{(i)} - \hat{h}_{\mu j}(x^{(i)}) \right)^2.$$

- (c) Update: $\hat{\mu}^{[m]} = \hat{\mu}^{[m-1]} + \text{sl} \cdot \hat{h}_{\mu j_\mu^*}(x)$.

Else: set $\hat{\mu}^{[m]} := \hat{\mu}^{[m-1]}$.

4. **Update** σ (**using** $\hat{\mu}^{[m]}$): If $m \leq m_{\text{stop}, \sigma}$:

- (a) Compute $u_\sigma^{(i)} = \frac{(y^{(i)} - \hat{\mu}^{[m]}(x^{(i)}))^2}{\hat{\sigma}^{[m-1]}(x^{(i)})^2} - 1$ for $i = 1, \dots, n$.

- (b) Fit each base learner $h_{\sigma j}(\cdot)$ to u_σ ; select

$$j_\sigma^* = \operatorname{argmin}_j \sum_{i=1}^n \left(u_\sigma^{(i)} - \hat{h}_{\sigma j}(x^{(i)}) \right)^2.$$

- (c) Update: $\hat{\eta}_\sigma^{[m]} = \hat{\eta}_\sigma^{[m-1]} + \text{sl} \cdot \hat{h}_{\sigma j_\sigma^*}(x)$, and set $\hat{\sigma}^{[m]} = \exp(\hat{\eta}_\sigma^{[m]})$.

Else: set $\hat{\eta}_\sigma^{[m]} := \hat{\eta}_\sigma^{[m-1]}$ (and $\hat{\sigma}^{[m]} := \hat{\sigma}^{[m-1]}$).

common stopping iteration or separate stopping iterations for the individual distribution parameters.

The two parameters can be updated by either a cyclic or a non-cyclic scheme. Algorithm 1 describes the cyclic approach: During each iteration, first the location is updated, followed by the scale, using the just-updated location estimate.

In contrast, the non-cyclic Algorithm 2, introduced by Thomas et al. (2018), only updates one predictor per iteration: First, for each distribution parameter, the best-performing base learner is selected based on the inner loss. Since the inner loss is not directly comparable across different distribution parameters, the outer loss is then used to compare the potential improvement from updating each parameter. The base learner that yields the lowest outer loss after the tentative update is then selected.

Thomas et al. (2018) mention two advantages that come with non-cyclic boosting: First, it does not require the use of separate **mstop** values for each parameter – **mstop** is scalar. This means that the ratio of μ and σ iterations

is chosen internally, so tuning `mstop` reduces to a one-dimensional problem, solvable via standard cross-validation, rather than a two-dimensional grid search. Second, non-cyclic boosting converges empirically faster than the cyclic algorithm.

Following the approach of `gamboostLSS` (Hofner et al., 2016), we implement both the cyclic and non-cyclic updating schemes in our R-package. This also allows us to empirically compare the two algorithms' predictive and variable-selection performance in our simulation study (Section 4).

Algorithm 2: Non-cyclic component-wise gradient boosting for GLSMs. Adapted from Thomas et al. (2018).

Initialization

1. Initialize $\hat{\eta}_\mu^{[0]}$ (identity link) and $\hat{\eta}_\sigma^{[0]}$ (log link) with offset values; set $\hat{\mu}^{[0]} = \hat{\eta}_\mu^{[0]}$ and $\hat{\sigma}^{[0]} = \exp(\hat{\eta}_\sigma^{[0]})$.
2. Specify base learners $h_{\mu 1}(\cdot), \dots, h_{\mu J_\mu}(\cdot)$ and $h_{\sigma 1}(\cdot), \dots, h_{\sigma J_\sigma}(\cdot)$.

For $m = 1$ **to** m_{stop} :

3. **For** $\theta \in \{\mu, \sigma\}$:

- (a) Compute $u_\mu^{(i)} = \frac{y^{(i)} - \hat{\mu}^{[m-1]}(x^{(i)})}{\hat{\sigma}^{[m-1]}(x^{(i)})^2}$ or $u_\sigma^{(i)} = \frac{(y^{(i)} - \hat{\mu}^{[m-1]}(x^{(i)}))^2}{\hat{\sigma}^{[m-1]}(x^{(i)})^2} - 1$ for $i = 1, \dots, n$.
- (b) Fit each base learner $h_{\theta j}(\cdot)$ to u_θ ; select $j_\theta^* = \operatorname{argmin}_j \sum_{i=1}^n \left(u_\theta^{(i)} - \hat{h}_{\theta j}(x^{(i)})\right)^2$.
- (c) Compute the potential improvement in the outer loss: $\Delta\ell_\theta = \sum_{i=1}^n \ell\left(y^{(i)}, \hat{\eta}^{[m-1]}(x^{(i)}) + \text{sl} \cdot \hat{h}_{\theta j_\theta^*}(x^{(i)})\right)$.

4. **Update** only the component $\theta^* = \operatorname{argmin}_{\theta \in \{\mu, \sigma\}} \Delta\ell_\theta$ with the larger improvement:

If $\theta^* = \mu$: $\hat{\mu}^{[m]} = \hat{\mu}^{[m-1]} + \text{sl} \cdot \hat{h}_{\mu j_\mu^*}(x)$, and set $\hat{\eta}_\sigma^{[m]} := \hat{\eta}_\sigma^{[m-1]}$
 $(\hat{\sigma}^{[m]} := \hat{\sigma}^{[m-1]})$.

If $\theta^* = \sigma$: $\hat{\eta}_\sigma^{[m]} = \hat{\eta}_\sigma^{[m-1]} + \text{sl} \cdot \hat{h}_{\sigma j_\sigma^*}(x)$, set $\hat{\sigma}^{[m]} = \exp(\hat{\eta}_\sigma^{[m]})$, and set $\hat{\mu}^{[m]} := \hat{\mu}^{[m-1]}$.

3 Usage and implementation

3.1 Usage

The function **boost_lmls** is the main function of the package and provides an interface for fitting GLSMs via gradient boosting. Table 1 lists all available arguments and their default values.

Argument	Default
<code>formula_mu</code>	—
<code>formula_sigma</code>	<code>= formula_mu</code>
<code>data</code>	—
<code>mstop</code>	—
<code>nu_mu</code>	0.1
<code>nu_sigma</code>	0.1
<code>method</code>	"non-cyclic"
<code>learner</code>	"linear"
<code>spline_degree</code>	3
<code>spline_nknots</code>	5
<code>spline_knot_strategy</code>	"quantile"
<code>cv</code>	FALSE
<code>k</code>	5
<code>progress</code>	FALSE

Table 1: Arguments and default values of `boost_lmls()`.

The model is specified by formulas for the location- and scale-parameters of the form `response ~ predictor1 + ... + predictor_i`. The boosting method is selected via `method`, while `nu_mu`, `nu_sigma`, and `mstop` control the step lengths and number of boosting iterations, respectively. For cyclical boosting, separate stopping iterations for the two parameters can be provided.

Base learners are selected via `learner`, with linear and spline available (see Section 3.2). The spline-specific arguments `spline_degree`, `spline_nknots`, and `spline_knot_strategy` control the spline specification. Setting `cv = TRUE` enables cross-validation for the selection of `mstop`, with `k` specifying the number of folds.

`progress` prints the selected base learner(s) for each iteration to the console.

Listing 1: Fitting a location-scale model with **boost_lmls**

```
fit <- boost_lmls(  
  formula_mu = y ~ x1 + x2 + x3,
```

```

    data = data,
    mstop = 100
)

```

Listing 1 shows a minimal call using only required arguments. All remaining arguments retain their default values, discussed in Section 3.2.

The **boost_lmls** function returns an object of S3 class “**boost_lmls**” which can be used with the S3 methods **predict()**, **plot()**, and **summary()**; exemplary outputs can be found in A.1.

Additionally, **tune_lmls** (see A.1 for a minimal use case) enables tuning of not only the number of iterations **mstop**, but also the step length ν , based on cross-validation. Like **boost_lmls** formulas for μ and σ , method, learner and number of folds for cross validation can be chosen. Furthermore, the user specifies the maximum number of boosting iterations using **mstop_max**, the range of candidate values for ν using **nu_min** and **nu_max**, and the number of candidate values within this range using **n_nu**.

3.2 Implementation

Architecture In order to ensure easier extensions and robust testing we generally aimed to follow a modular structure while building the package. The main function coordinates the overall fitting process while individual tasks are handled by dedicated helper functions. Furthermore, to keep the implementation self-contained there are no third-party dependencies, external packages were used for testing only.

The main function **boost_lmls()** first processes the supplied formulas and data to construct the design matrices for the location and scale predictors. Intercepts generated by the formulas are removed, as they are not used in the boosting procedure. To ensure faster convergence, the model is initialized with constant predictors based on the sample mean and standard deviation, $\hat{\mu}^{[0]} = \bar{y}$ and $\hat{\eta}_{\sigma}^{[0]} = \log(\text{sd}(y))$. The corresponding intercept values, $\beta_0^{[0]} = \bar{y}$ and $\gamma_0^{[0]} = \log(\text{sd}(y))$, are stored in the model object, while all remaining coefficients are initialized at zero.

If requested, early stopping (see below) is performed before the main boosting procedure. Depending on the selected method, the cyclic or non-cyclic algorithm described in Section 2.2 is then executed, with the parameter updates delegated to dedicated update functions. The resulting model information is collected in an object of class **boost_lmls**, which is used by the corresponding S3 methods.

Base Learners Currently, the package supports two different base learners for fitting continuous data. Linear effects can be estimated with linear learners, which rely on two degrees of freedom: the corresponding covariate x_i and the intercept. Fitting with an intercept avoids the slow or incorrect convergence that results from forcing the effect through the origin, while keeping the base learner’s degrees of freedom low helps prevent overshooting (Hofner et al., 2012).

For non-linear covariate effects, B-spline base learners were implemented (in contrast to penalized spline base learners, no additional smoothing penalty is applied). For a spline of degree (d) with (K) distinct internal knots, the corresponding base learner contains (K+d) B-spline basis functions and an additional intercept, resulting in (K+d+1) degrees of freedom (Fahrmeir et al., 2013). Internal knots can be placed either at equally spaced locations over the covariate range or at empirical quantiles. The latter adapts the knot locations to the observed predictor distribution and avoids allocating excessive flexibility to regions containing comparatively few observations. Under the default specification of cubic splines with five internal knots, this corresponds to nine degrees of freedom when all knots are distinct. The complexity of an individual spline learner is determined by the spline degree and number of knots, while the learning rate and early stopping regularize the overall boosting procedure.

Early Stopping To avoid overfitting we implemented an early stopping method. In this context Hofner et al. (2012) advise to tune the number of iterations `mstop` via k -fold cross-validation. While this is computationally feasible in the scalar case of non-cyclic boosting it quickly becomes too costly in a dense, two-dimensional grid in the cyclic case.

For the latter, we therefore chose a reduced one: For $\text{mstop}_\mu \geq \text{mstop}_\sigma$, the grid is dense in mstop_μ and sparse in mstop_σ , meaning that every candidate value for mstop_μ is evaluated, whereas mstop_σ is restricted to a small set of candidates on a logarithmic scale. For $\text{mstop}_\mu < \text{mstop}_\sigma$, candidate values for both parameters are restricted to a logarithmic grid. This follows the approach described by Hofner et al. (2016), who argue μ is usually “of greatest interest (...) and thus more care should be taken to accurately model this parameter” than σ . For a visualization see Figure A2 in the Appendix.

Default Values To enhance usability, default values were provided where appropriate; Table 1 lists the selected values. While mandatory input arguments such as `formula_mu` and `data` inherently lack defaults, several choices warrant further explanation: `formula_sigma` mirrors `formula_mu`, so that

users don't need to specify a separate model for the scale parameter when no prior assumption about which covariates drive heteroskedasticity is available. Since the maximal number of iterations `mstop` depends on both number of observations and predictors no default is set. To ensure fast computability and for the advantages mentioned before, we chose non-cyclic boosting with linear base learners. Cross-validation is disabled (`cv = FALSE`) due to its higher computational cost, with 5-fold cross-validation recommended when used. Following standard recommendations in the literature, the step length is set to 0.1 for both parameters (Hofner et al., 2016; Bühlmann & Hothorn, 2007).

For spline base learners, cubic B-splines (`spline_degree = 3`) with five internal knots (`spline_nknots = 5`) are used by default. Cubic splines provide a conventional compromise between smoothness and flexibility. Since the implemented spline base learners are unpenalized, their complexity increases directly with the number of basis functions (Fahrmeir et al., 2013). Five internal knots therefore provide a pragmatic moderate-complexity starting point; together with the default cubic degree, the resulting learner has nine degrees of freedom when all internal knots are distinct. These settings are intended as practical starting values and can be modified when more or less flexible effects are required.

Testing We tested for the functionality of our package through unit tests implemented with `testthat` (Wickham, 2009). The `boost_lmls`-package passes R CMD `check` with zero errors, warnings, or notes. Test coverage was assessed using `covr` (Hester, 2015), which indicates that the implemented tests exercise 100% of the package's code (see A7 in the Appendix).

4 Simulation study

4.1 Simulation Study Design

The following simulation study aims to empirically evaluate the proposed gradient boosting approach for location and scale regression. It is structured according to the ADEMP framework proposed by Morris et al. (2019), which distinguishes between Aims, Data-generating mechanisms, Estimands and other targets, Methods, and Performance measures. The technical implementation follows the simulation framework of Hofert & Mächler (2016), separating scenario specification, data generation, model fitting, and the evaluation and aggregation of simulation results.

Aims

- Assess whether the proposed boosting implementation, evaluated under controlled data-generating processes, correctly
 - recovers the underlying location and scale effects
 - identifies informative covariates
- Compare the performance of cyclic and non-cyclic boosting and characterize potential differences in convergence behavior
- Provide an exploratory assessment of whether spline base learners improve recovery of nonlinear location and scale effects, relative to linear learners

Data-generating mechanisms

For all simulation scenarios, data are generated according to the Gaussian location-scale model introduced in Section 2,

$$Y_i \mid \mathbf{x}_i \sim \mathcal{N} \left(f_\mu(\mathbf{x}_i), [\exp \{f_\sigma(\mathbf{x}_i)\}]^2 \right), \quad i = 1, \dots, n.$$

For each observation, the covariate vector is generated as

$$\mathbf{x}_i \stackrel{\text{iid}}{\sim} \mathcal{N}_p(\mathbf{0}, \boldsymbol{\Sigma}_{\rho_X}), \quad i = 1, \dots, n,$$

where $\boldsymbol{\Sigma}_{\rho_X}$ has unit variances and constant pairwise correlation ρ_X , that is,

$$(\boldsymbol{\Sigma}_{\rho_X})_{jk} = \begin{cases} 1, & j = k, \\ \rho_X, & j \neq k. \end{cases}$$

The data-generating processes (DGPs) are designed to represent different characteristics of the regression problem, including balanced and imbalanced location-scale signals, sparsity, and nonlinear effects. Table 2 summarizes the individual settings, the number of covariates, and the corresponding predictor functions for location and log-scale.

Table 2: Overview of the data-generating processes.

DGP	Setting	p	$f_\mu(\mathbf{x})$	$f_\sigma(\mathbf{x})$
1	Linear balanced	5	$x_1 + x_2$	$0.3x_3 + 0.3x_4$
2a	Location-dominant	5	$1.5x_1 + 1.5x_2$	$0.1x_3$
2b	Scale-dominant	5	$0.25x_1$	$0.4x_2 + 0.4x_3$
3	Sparse	20	$x_1 + x_2$	$0.3x_3 + 0.3x_4$
4	Nonlinear	5	$2\sin(\pi x_1) + x_2^2$	$0.3\sin(\pi x_3)$

Note: DGP 3, $x_5, \dots, x_{20}$ are non-informative covariates.

Since the scale parameter is linked to the additive predictor through $\sigma_i = \exp\{f_\sigma(\mathbf{x}_i)\}$, comparatively smaller coefficients are chosen for the scale predictor to avoid unrealistically large variations in the conditional standard deviation.

Table 3: Overview of the factors varied across the simulation scenarios.

Factor	Values	Purpose
Sample size n	100, 500, 1000	Assesses the performance of the boosting algorithms under increasing amounts of information.
Predictor correlation ρ_X	0, 0.5	Investigates the robustness of the boosting algorithms when predictors are moderately correlated.

The simulation study comprises two parts. The main study combines DGPs 1, 2a, 2b, and 3 with the three sample sizes and two correlation levels, resulting in 24 scenarios. A separate study of nonlinear effects considers DGP 4 under the same sample sizes and correlation levels, adding six scenarios. Together, the two parts comprise 30 simulation scenarios.

Estimands

The primary estimands of the simulation study are the conditional location and scale functions of the Gaussian location-scale model,

$$\begin{aligned}\mu(\mathbf{x}) &= f_\mu(\mathbf{x}), \\ \sigma(\mathbf{x}) &= \exp\{f_\sigma(\mathbf{x})\}.\end{aligned}$$

Since both functions are explicitly specified by the respective data-generating process, their true values are known and can be compared with the corresponding estimates obtained from the fitted boosting models.

In addition, the sets of covariates with a true effect on the location and scale parameters are considered as targets of the simulation study. We denote these sets by S_μ and S_σ , respectively. This is particularly relevant for the sparse setting, in which informative and non-informative covariates are known by construction.

Methods

The methods considered in the simulation study are the cyclic and non-cyclic component-wise gradient boosting algorithms introduced in Section 2. The comparison of both updating schemes is motivated by Thomas et al. (2018), who proposed the non-cyclic approach for boosted distributional regression as an alternative to conventional cyclic updating.

For DGPs 1–3, both methods are fitted using linear base-learners, resulting in a direct comparison of the cyclic and non-cyclic updating schemes. For DGP 4, the non-cyclic method is fitted with linear and spline base-learners. Holding the updating scheme fixed isolates the effect of base-learner flexibility.

For all boosted models, the learning rates are fixed at $\nu_\mu = \nu_\sigma = 0.1$. Model complexity is controlled by early stopping, with the stopping iteration selected using five-fold cross-validation based on the Gaussian negative log-likelihood. For DGPs 1–3, we limited the number of boosting iterations to 4,000 for each of the location and scale models in the cyclic approach for computational reasons, whereas, following (Thomas et al., 2018), the non-cyclic algorithm was allowed up to $2 \times 4,000 = 8,000$ iterations. Since a pilot study indicated that DGP 4 required fewer iterations, the maximum number of iterations was reduced to 800 for this part of the study.

As an additional reference, DGPs 1–3 are also fitted via maximum-likelihood using `gamlss` (Rigby & Stasinopoulos, 2005) with the Gaussian family. These fits provide two benchmarks rather than competing methods: a *full* fit including all predictors in both equations, representing the practitioner’s default in the absence of a sparsity method, and an *oracle* fit including only the true active predictors, representing the best achievable estimation accuracy given knowledge of the true model. Since boosting algorithms converge towards the maximum-likelihood estimate as `mstop` increases (Mayr & Hofner, 2018), the *full* fit additionally serves as a reference point for the boosting estimates.

Performance Measures

The performance measures are chosen according to the estimands and targets defined above. All predictive performance measures are evaluated on an independently generated test data set in order to assess out-of-sample performance.

Recovery of the location and scale functions is assessed using the root mean squared error (RMSE). RMSE expresses estimation error on the same scale as the corresponding target function, facilitating interpretation compared with MSE, while retaining the emphasis on larger deviations inherent in squared-error measures. For the scale component, the RMSE is evaluated on the linear predictor scale,

$$\eta_\sigma = \log(\sigma),$$

which is consistent with the model specification $\sigma = \exp(\eta_\sigma)$.

To assess the joint predictive performance of the location and scale estimates, we additionally consider the average Gaussian negative log-likelihood (NLL),

$$\text{NLL} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \left[\log(\hat{\sigma}_i) + \frac{(y_i - \hat{\mu}_i)^2}{2\hat{\sigma}_i^2} \right].$$

The additive constant of the Gaussian negative log-likelihood is omitted, as it does not affect comparisons between methods.

Variable-selection performance is assessed using the true positive rate (TPR) and false positive rate (FPR), calculated separately for the location and scale components by comparing the selected covariates with the corresponding true active sets S_μ and S_σ .

Lower values of the RMSE measures and the NLL indicate better predictive performance, whereas higher TPR and lower FPR indicate better variable-selection performance. Formal definitions of the RMSE, TPR, and FPR measures are provided in Appendix A.1.

Computational implementation

The computational implementation of the simulation study is guided by Hofert & Mächler (2016) and the coding and execution recommendations of Morris et al. (2019). In particular, the specification of simulation scenarios, data generation, model fitting, and the aggregation and analysis of simulation results are implemented as separate components.

4.2 Simulation Study Results

Each method and scenario combination was evaluated in 500 Monte Carlo replications. The comparison of cyclic and non-cyclic boosting with linear base-learners across DGPs 1, 2a, 2b, and 3 therefore comprised 24,000 model-fitting runs (see Appendix Tables A2 – A8), while the comparison of linear and spline base-learners using non-cyclic boosting for DGP 4 comprised another 6,000 model-fitting runs (see Appendix Table A9 for detailed results). Unless stated otherwise, the summaries below retain every finite run, including runs with diagnostic warnings; only fits with non-finite results and status **error** are excluded. Two such errors occurred in the linear study, both for cyclic boosting in DGP 3 with $n = 100$ and $\rho_X = 0$, whereas no fit failed in the nonlinear study. These failures resulted from numerical instability during the location update, where extreme variance estimates caused unstable location gradients. Both failures occurred in the smallest-sample scenario ($n = 100$), consistent with the general observation that model estimates tend to become less stable as the sample size decreases (Vey et al., 2026). Nevertheless, because only two of the 24,000 linear fits were affected, they were treated as isolated numerical exceptions.

Among the retained warnings, the most frequent cause was the boosting iteration limit being reached for at least one parameter, this occurred in 0.6% of cyclic and 0.02% of non-cyclic fits for DGPs 1–3, and in no fits for DGP 4. All figures supporting the following results are provided in Appendix Figures A3 – A6. The reported means are accompanied by 95% Monte Carlo t -intervals to quantify simulation uncertainty (Morris et al., 2019). Because the NLL values of the spline fits were extremely right-skewed, typical performance for DGP 4 is summarized using the median and interquartile range. The replication-level NLL distribution is shown on a logarithmic scale in Appendix Figure A6.

Recovery of linear location and scale effects Across all simulation settings both boosting methods seem to recover the linear location and scale effects reasonably well. For sample sizes below $n = 1000$, boosting generally achieves lower RMSE and NLL (Appendix Tables A2, A3 and A4) than the unregularized maximum-likelihood estimator, with the largest differences observed for the sparse DGP 3. This result suggests that our package offers (the expected) boosting advantages in settings where the estimation problem is more challenging or the underlying structure is sparse. However, further experiments in genuinely high-dimensional settings are required to assess whether this advantage persists as the dimensionality increases substantially.

For $n = 1000$, the regularized maximum-likelihood estimator occasion-

ally outperforms boosting, indicating that the relative performance of the two approaches depends on the sample size and data-generating process. Nevertheless, boosting consistently falls short of the oracle fit, an expected pattern, since the oracle uses the true active predictors without having to estimate which variables are relevant.

Furthermore, in line with previous boosting studies (Vey et al., 2026; Umlauf et al., 2023), increasing the sample size improves both estimation accuracy and predictive performance, as reflected in decreasing RMSE and NLL values.

Variable selection Throughout the linear study both boosting methods achieved high TPR for both parameters (Appendix Tables A5 and A6), with power dropping slightly below 1.0 only in the sparser or smaller-sample scenarios (DGP 2b and DGP 3 at $n = 100$).

The results for the FPR (Appendix Tables A7 and A8), however, are more nuanced: They varied substantially, ranging from below 0.08 up to 0.97 depending on DGP, sample size and parameter. Rates tend to be higher for the scale component than for the location and do not decrease monotonically with sample size in most scenarios. While not ideal, the inclusion of too many variables is a known issue in the boosting literature and, as shown here, does not appear to substantially affect predictive performance (Strömer et al., 2022).

Cyclic vs. non-cyclic boosting When comparing cyclic and non-cyclic boosting one important limitation has to be noted: As discussed earlier the optimization of cyclic fits relies on a sparse grid, which is a potential disadvantage, especially when comparing the convergence speed.

In order to evaluate the latter, we compare, in line with Thomas et al. (2018), the sum of the parameter updates for the cyclic updating scheme with number of iterations used in the non-cyclic algorithm. Table 4 shows that the non-cyclic algorithm generally requires fewer parameter updates until convergence than the cyclic algorithm, suggesting faster convergence.

Table 4: Number of parameter updates until convergence.

Algorithm	Min	Q1	Median	Mean	Q3	Max
Cyclic	2.0	117.0	201.0	249.5	221.0	8000.0
Non-cyclic	1.0	85.0	134.0	132.4	166.0	8000.0

In terms of recovering the underlying location both updating schemes perform similarly: The RMSE_μ (Table A2) differ only slightly with no clear

direction. $\text{RMSE}_{\eta\sigma}$ (Appendix Table A3), however, exhibits a different pattern: Here the cyclic algorithm consistently underperforms relative to non-cyclic. Further investigations are needed in order to evaluate whether this is caused by the algorithm itself or through the search grid which is sparse in σ . However, the observed differences are still rather small as the NLL (Appendix Table A4) returns almost identical values for both methods.

Looking at the TPR (Appendix Tables A5 and A6) both methods are essentially indistinguishable: As mentioned above power is close to 1.0 for almost all scenarios and behaves similarly for both algorithms.

The clearest and most consistent difference between the two variants is however the FPR (Appendix Tables A7 and A8): Across all DGPs, correlations and sample sizes and parameters non-cyclic achieves a much lower FPR. The gap between the two updating schemes can be substantial: Most notably for DGP 1 with $n = 500$, cyclic boosting’s FPR for the scale component exceeds that of non-cyclic boosting by 37.7 percentage points, despite comparable predictive accuracy between the two methods in this scenario.

Overall, the comparison of the two updating schemes shows that, while achieving comparable predictive performance, non-cyclic boosting on average yields a lower FPR and requires fewer parameter updates. This is in line with the findings of Thomas et al. (2018) and suggests that non-cyclic boosting is the generally preferable choice.

Comparison of linear and spline base-learners DGP 4 provides an exploratory assessment of whether the additional flexibility of spline base-learners improves recovery of nonlinear location and scale effects relative to linear base-learners. Figure A5 illustrates the overall performance patterns, while Table A9 reports the corresponding numerical summaries; both are provided in the Appendix.

The results show a clear sample-size dependence, with qualitatively similar patterns for $\rho_X = 0$ and $\rho_X = 0.5$. At $n = 100$, the spline provides little advantage: the RMSE_μ is similar for both learners, while the $\text{RMSE}_{\eta\sigma}$ is slightly larger for the spline (0.925 versus 0.824). With increasing sample size, however, the spline increasingly benefits from its ability to represent the nonlinear effects. At $n = 1000$, it reduces RMSE_μ from 2.002 to 1.145 and $\text{RMSE}_{\eta\sigma}$ from 0.84 to 0.39. Joint predictive performance shows the same qualitative pattern, with the median NLL decreasing from 1.32 for the linear learner to 0.70 for the spline at $n = 1000$.

The spline fits also revealed some numerical instability. Although all 3,000 spline fits completed without an error, warnings for extreme log-scale predictions or observation-level NLL contributions occurred in 27.4% of spline

fits for $\rho_X = 0$ and 30.8% for $\rho_X = 0.5$ at $n = 100$, decreasing to 2.6% and 3.4%, respectively, at $n = 1000$; see Figure A6.

Overall, the exploratory comparison suggests that spline base-learners can substantially improve recovery of nonlinear location and scale effects once sufficient data are available, while providing little advantage at small sample sizes. The remaining numerical instability motivates further investigation of stronger regularization and more robust extrapolation strategies. Penalized P-splines, which combine B-spline bases with an explicit smoothness penalty, therefore constitute a natural direction for extending the implementation (Eilers & Marx, 1996).

5 Conclusions

In this work, we introduced `boost.lmls`, an R package designed for component-wise gradient boosting GLSM. The package supports cyclic and non-cyclic update strategies, includes linear- and spline-base-learners and provides an early stopping method via cross-validation.

The simulation study demonstrates that our implementation is able to reliably recover both location and scale effects for the linear case. Furthermore, the package proved effective in selecting informative covariates. While the FPR is quite high it did not affect prediction quality, implying that the false coefficients are rather small, which is not uncommon for boosting algorithms (Strömer et al., 2022). The comparison of cyclic and non-cyclic boosting showed that the latter is generally favorable: it requires fewer parameter updates and has better FPR, even though prediction accuracy is comparable. Finally, we showed that for growing sample sizes our spline-base-learner is capable of significantly outperforming linear-base-learners when dealing with nonlinear location and scale effects.

Several limitations remain. First, the behavior of the implementation in genuinely high-dimensional settings ($p > n$) has not yet been investigated. Second, the cyclic boosting approach was evaluated on a (semi-)sparse search grid for computational reasons. This restriction may have affected the comparison between cyclic and non-cyclic boosting. A more comprehensive evaluation using an improved early stopping strategy would therefore be valuable. Third, a systematic comparison with established boosting implementations, such as `gamboostLSS`, is needed to assess the computational and predictive performance of `boost.lmls` relative to existing methods. Finally, all results are based on simulated data, and an application to real-world data remains an important next step. Similarly, a dedicated simulation study evaluating the behavior of `tune.lmls` was beyond the scope of this work.

Future extensions of the package could contain adding different distribution families and base learners, such as trees. Additionally, formal inference tools, such as the significance tests proposed by Hepp et al. (2019) could be included.

Bibliography

- Bühlmann, P. & Hothorn, T. (2007). Boosting Algorithms: Regularization, Prediction and Model Fitting. *Statistical Science*, 22(4).
- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). San Francisco California USA: ACM.
- Eilers, P. H. C. & Marx, B. D. (1996). Flexible smoothing with b-splines and penalties. *Statistical Science*, 11(2), 89–121.
- Fahrmeir, L., Kneib, T., Lang, S., & Marx, B. (2013). *Regression: Models, Methods and Applications*. Berlin, Heidelberg: Springer Berlin Heidelberg.
- Hepp, T., Schmid, M., & Mayr, A. (2019). Significance Tests for Boosted Location and Scale Models with Linear Base-Learners. *The International Journal of Biostatistics*, 15(1), 20180110.
- Hester, J. (2015). Covr: Test Coverage for Packages.
- Hofert, M. & Mächler, M. (2016). Parallel and Other Simulations in R Made Easy: An End-to-End Study. *Journal of Statistical Software*, 69(4).
- Hofner, B., Mayr, A., Robinzonov, N., & Schmid, M. (2012). Model-based Boosting in R: A Hands-on Tutorial Using the R Package mboost.
- Hofner, B., Mayr, A., & Schmid, M. (2016). **gamboostLSS** : An R Package for Model Building and Variable Selection in the GAMLSS Framework. *Journal of Statistical Software*, 74(1).
- Mayr, A., Fenske, N., Hofner, B., Kneib, T., & Schmid, M. (2010). GAMLSS for high-dimensional data - a flexible approach based on boosting.
- Mayr, A., Fenske, N., Hofner, B., Kneib, T., & Schmid, M. (2012). Generalized Additive Models for Location, Scale and Shape for High Dimensional Data—A Flexible Approach Based on Boosting. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 61(3), 403–427.
- Mayr, A. & Hofner, B. (2018). Boosting for statistical modelling-A non-technical introduction. *Statistical Modelling*, 18(3-4), 365–384.

- Morris, T. P., White, I. R., & Crowther, M. J. (2019). Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11), 2074–2102.
- Rigby, R. A. & Stasinopoulos, D. M. (2005). Generalized Additive Models for Location, Scale and Shape. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 54(3), 507–554.
- Strömer, A., Staerk, C., Klein, N., Weinhold, L., Titze, S., & Mayr, A. (2022). Deselection of base-learners for statistical boosting—with an application to distributional regression. *Statistical Methods in Medical Research*, 31(2), 207–224.
- Thomas, J., Mayr, A., Bischl, B., Schmid, M., Smith, A., & Hofner, B. (2018). Gradient boosting for distributional regression: Faster tuning and improved variable selection via noncyclical updates. *Statistics and Computing*, 28(3), 673–687.
- Umlauf, N., Seiler, J., Wetscher, M., Simon, T., Lang, S., & Klein, N. (2023). Scalable Estimation for Structured Additive Distributional Regression.
- Vey, J. A., Heinze, G., & Kieser, M. (2026). Variable selection for clinical prediction models in low-dimensional data - a simulation study comparing traditional regression and machine learning methods. *BMC Medical Research Methodology*, 26(1), 151.
- Wickham, H. (2009). Testthat: Unit Testing for R.
- Zhang, B., Hepp, T., Greven, S., & Bergherr, E. (2021). Adaptive Step-Length Selection in Gradient Boosting for Generalized Additive Models for Location, Scale and Shape.

A Appendix

A.1 Additional Figures and Tables

S3 methods

Listing A1: Example output of `summary()` for a `boost_lmls` object

```
Call: boost_lmls(formula_mu = y ~ x1 + x2 + x3, data =
  data,
  mstop = 100)

Families:
Normal distribution: mu (identity link)
Loss function: -dnorm(x = y, mean = f, sd = sigma, log =
  TRUE)

Normal distribution: sigma (log link)
Loss function: -dnorm(x = y, mean = mu, sd = exp(f), log
  = TRUE)

Number of iterations (mstop): mu = 100 sigma = 100

Learning rates: mu = 0.1 sigma = 0.1

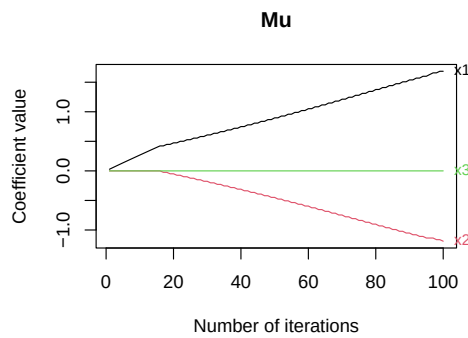
Selection frequencies:
  mu:
    x1    x2    x3
0.58 0.42 0.00

  sigma:
    x2    x3    x1
0.60 0.38 0.02

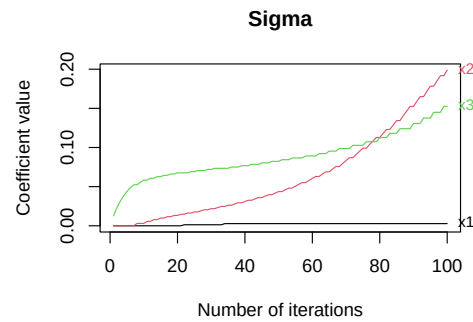
Coefficients:
  mu:
Intercept          x1          x2          x3
1.127735  1.685177 -1.189615  0.000000

  sigma:
Intercept          x1          x2          x3
0.081254173 0.002750888 0.198765327 0.152529198
```

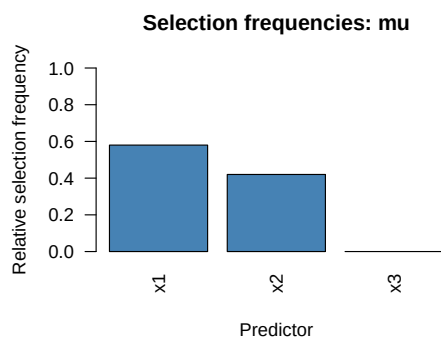
Pseudo R-squared:	
Cox & Snell:	0.9397
Nagelkerke:	0.9450



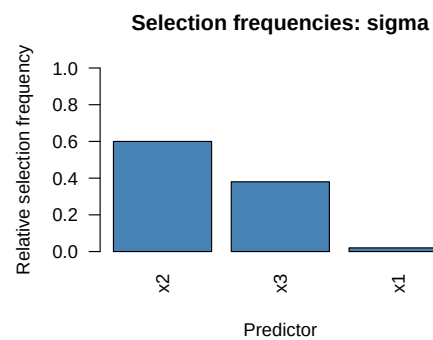
(a)



(b)



(c)



(d)

Figure A1: Graphical output of the `plot()` method for a fitted `boost.lmls` model.

Listing A2: Example of a prediction using the `predict()` method

```
newdata <- data.frame(x1 = 3, x2 = 0.5, x3 = -1)

predict(fit, newdata = newdata, parameter = "mu", type =
  "response")
      [,1]
[1,] 5.588459
```

`tune_lmls`

Listing A3: Tuning step length and stopping iterations with `tune_lmls`

```
fit_tune <- tune_lmls(  
  formula_mu = y ~ x1 + x2 + x3  
  data = data  
  mstop_max = 200  
  nu_min = 0.01  
  nu_max = 0.5  
  n_nu = 10  
)
```

Early stopping

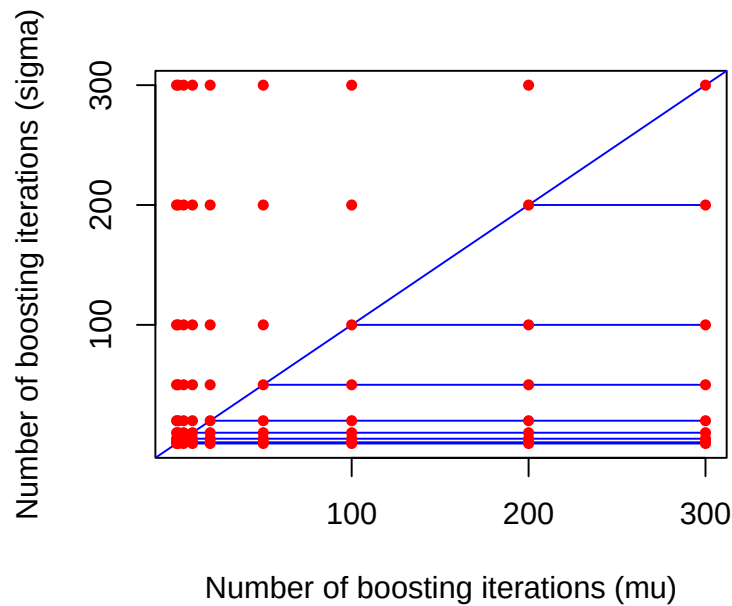


Figure A2: Illustration of the reduced two-dimensional grid used for tuning $m_{\text{stop},\mu}$ and $m_{\text{stop},\sigma}$ via cross-validation. Red points mark stopping combinations that require a dedicated model fit; blue lines indicate that, once a stopping iteration is fixed on one axis (drawn from the reduced grid), the corresponding boosting path is evaluated at every iteration along the other axis without requiring additional model fits, since the full path is already available from that single fit. Own illustration, following the visualization style of Hofner et al. (2016).

ADEMP Framework

Table A1: Overview of the ADEMP framework based on Morris et al. (2019).

Component		Definition
A	Aims	Specifies the aims and research questions of the simulation study.
D	Data-generating mechanisms	Defines how the data are generated and which factors and scenarios are varied.
E	Estimand / target of analysis	Specifies the estimands or other targets that are investigated in the simulation study.
M	Methods	Defines the statistical methods to be evaluated and, where applicable, compared.
P	Performance measures	Specifies the measures used to evaluate the performance of the considered methods.

Definition of Performance Measures

The recovery of the true location function is measured by

$$\text{RMSE}_\mu = \sqrt{\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (\hat{\mu}_i - \mu_i)^2}, \quad (1)$$

where $\mu_i = f_\mu(\mathbf{x}_i)$ denotes the true conditional location and $\hat{\mu}_i$ the corresponding estimate.

For the scale component, let

$$\eta_{\sigma,i} = f_\sigma(\mathbf{x}_i) = \log(\sigma_i). \quad (2)$$

The corresponding RMSE on the linear predictor scale is

$$\text{RMSE}_{\eta_\sigma} = \sqrt{\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (\hat{\eta}_{\sigma,i} - \eta_{\sigma,i})^2}. \quad (3)$$

For each distribution parameter, variable-selection performance is measured using the true positive rate and false positive rate,

$$\text{TPR} = \frac{\{\text{selected active covariates}\}}{\{\text{active covariates}\}}, \quad (4)$$

$$\text{FPR} = \frac{\{\text{selected inactive covariates}\}}{\{\text{inactive covariates}\}}. \quad (5)$$

Simulation Study Tables DGP 1–3

Table A2: Recovery performance RMSE_μ , by DGP, n and ρ_X .

dgp	n	rho_x	cyclic	gamlss-full	gamlss-oracle	non-cyclic
1	100	0	0.220 (0.084)	0.238 (0.082)	0.140 (0.064)	0.217 (0.089)
1	100	0.5	0.205 (0.079)	0.218 (0.075)	0.136 (0.066)	0.219 (0.097)
1	500	0	0.084 (0.030)	0.096 (0.031)	0.060 (0.025)	0.084 (0.030)
1	500	0.5	0.082 (0.030)	0.092 (0.030)	0.060 (0.028)	0.083 (0.033)
1	1000	0	0.058 (0.021)	0.067 (0.021)	0.042 (0.017)	0.058 (0.021)
1	1000	0.5	0.054 (0.021)	0.062 (0.021)	0.041 (0.020)	0.056 (0.022)
2a	100	0	0.240 (0.091)	0.252 (0.083)	0.161 (0.069)	0.252 (0.100)
2a	100	0.5	0.231 (0.084)	0.256 (0.080)	0.166 (0.069)	0.250 (0.096)
2a	500	0	0.101 (0.033)	0.107 (0.031)	0.069 (0.027)	0.098 (0.034)
2a	500	0.5	0.098 (0.036)	0.108 (0.034)	0.072 (0.032)	0.097 (0.037)
2a	1000	0	0.072 (0.024)	0.076 (0.023)	0.051 (0.022)	0.070 (0.024)
2a	1000	0.5	0.067 (0.023)	0.074 (0.022)	0.050 (0.021)	0.067 (0.024)
2b	100	0	0.173 (0.083)	0.221 (0.085)	0.098 (0.053)	0.156 (0.074)
2b	100	0.5	0.181 (0.456)	0.198 (0.072)	0.094 (0.057)	0.151 (0.075)
2b	500	0	0.070 (0.032)	0.087 (0.030)	0.043 (0.022)	0.062 (0.029)
2b	500	0.5	0.064 (0.030)	0.079 (0.028)	0.039 (0.022)	0.063 (0.029)
2b	1000	0	0.052 (0.020)	0.060 (0.021)	0.029 (0.015)	0.042 (0.018)
2b	1000	0.5	0.052 (0.031)	0.054 (0.019)	0.028 (0.017)	0.047 (0.021)
3	100	0	0.311 (0.102)	0.826 (0.192)	0.140 (0.060)	0.311 (0.123)
3	100	0.5	0.287 (0.101)	0.775 (0.191)	0.138 (0.066)	0.312 (0.121)
3	500	0	0.109 (0.032)	0.187 (0.031)	0.059 (0.024)	0.105 (0.033)
3	500	0.5	0.106 (0.031)	0.175 (0.030)	0.058 (0.026)	0.108 (0.037)
3	1000	0	0.075 (0.022)	0.127 (0.022)	0.044 (0.018)	0.075 (0.024)
3	1000	0.5	0.070 (0.022)	0.116 (0.019)	0.040 (0.019)	0.074 (0.026)

Table A3: Recovery performance $\text{RMSE}_{\eta_\sigma}$, by DGP, n and ρ_X .

dgp	n	rho_x	cyclic	gamlss-full	gamlss-oracle	non-cyclic
1	100	0	0.206 (0.075)	0.217 (0.075)	0.123 (0.059)	0.169 (0.068)
1	100	0.5	0.210 (0.082)	0.221 (0.076)	0.130 (0.063)	0.175 (0.067)
1	500	0	0.073 (0.022)	0.078 (0.023)	0.051 (0.022)	0.067 (0.023)
1	500	0.5	0.075 (0.026)	0.079 (0.025)	0.052 (0.023)	0.067 (0.025)
1	1000	0	0.049 (0.015)	0.053 (0.015)	0.035 (0.014)	0.046 (0.016)
1	1000	0.5	0.047 (0.016)	0.052 (0.016)	0.035 (0.015)	0.045 (0.017)
2a	100	0	0.214 (0.087)	0.218 (0.071)	0.098 (0.053)	0.168 (0.057)
2a	100	0.5	0.197 (0.082)	0.213 (0.071)	0.097 (0.051)	0.156 (0.056)
2a	500	0	0.078 (0.024)	0.078 (0.025)	0.040 (0.021)	0.069 (0.024)
2a	500	0.5	0.073 (0.022)	0.080 (0.025)	0.041 (0.021)	0.066 (0.022)
2a	1000	0	0.056 (0.017)	0.054 (0.016)	0.029 (0.015)	0.047 (0.015)
2a	1000	0.5	0.049 (0.015)	0.054 (0.016)	0.029 (0.015)	0.045 (0.015)
2b	100	0	0.186 (0.069)	0.218 (0.076)	0.127 (0.055)	0.171 (0.072)
2b	100	0.5	0.174 (0.070)	0.214 (0.068)	0.122 (0.052)	0.168 (0.065)
2b	500	0	0.074 (0.026)	0.079 (0.024)	0.053 (0.022)	0.068 (0.026)
2b	500	0.5	0.078 (0.027)	0.079 (0.024)	0.052 (0.022)	0.066 (0.026)
2b	1000	0	0.047 (0.016)	0.054 (0.016)	0.035 (0.015)	0.047 (0.017)
2b	1000	0.5	0.047 (0.017)	0.053 (0.016)	0.035 (0.015)	0.045 (0.016)
3	100	0	0.333 (0.126)	1.871 (0.699)	0.124 (0.054)	0.250 (0.079)
3	100	0.5	0.294 (0.113)	1.860 (0.672)	0.130 (0.058)	0.237 (0.067)
3	500	0	0.121 (0.033)	0.176 (0.032)	0.053 (0.022)	0.088 (0.026)
3	500	0.5	0.113 (0.033)	0.175 (0.030)	0.052 (0.023)	0.087 (0.027)
3	1000	0	0.072 (0.016)	0.111 (0.019)	0.036 (0.016)	0.062 (0.018)
3	1000	0.5	0.068 (0.017)	0.112 (0.019)	0.036 (0.016)	0.060 (0.019)

Table A4: Recovery performance NLL, by DGP, n and ρ_X .

dgp	n	rho_x	cyclic	gamlss-full	gamlss-oracle	non-cyclic
1	100	0	0.582 (0.061)	0.626 (0.097)	0.540 (0.043)	0.574 (0.061)
1	100	0.5	0.583 (0.060)	0.626 (0.090)	0.544 (0.047)	0.576 (0.060)
1	500	0	0.512 (0.028)	0.515 (0.028)	0.507 (0.027)	0.511 (0.028)
1	500	0.5	0.509 (0.029)	0.512 (0.029)	0.504 (0.028)	0.508 (0.029)
1	1000	0	0.506 (0.028)	0.507 (0.028)	0.504 (0.027)	0.506 (0.027)
1	1000	0.5	0.505 (0.029)	0.506 (0.029)	0.503 (0.029)	0.505 (0.029)
2a	100	0	0.585 (0.059)	0.611 (0.082)	0.530 (0.034)	0.572 (0.051)
2a	100	0.5	0.579 (0.059)	0.608 (0.077)	0.531 (0.034)	0.566 (0.053)
2a	500	0	0.513 (0.024)	0.514 (0.025)	0.505 (0.023)	0.511 (0.024)
2a	500	0.5	0.511 (0.026)	0.513 (0.027)	0.504 (0.025)	0.509 (0.026)
2a	1000	0	0.505 (0.022)	0.505 (0.023)	0.502 (0.022)	0.504 (0.022)
2a	1000	0.5	0.505 (0.024)	0.506 (0.024)	0.502 (0.024)	0.504 (0.024)
2b	100	0	0.573 (0.061)	0.634 (0.114)	0.534 (0.042)	0.562 (0.059)
2b	100	0.5	1.265 (12.606) ^a	0.633 (0.100)	0.535 (0.047)	0.559 (0.052)
2b	500	0	0.510 (0.030)	0.513 (0.030)	0.505 (0.029)	0.509 (0.030)
2b	500	0.5	0.513 (0.033)	0.516 (0.033)	0.506 (0.032)	0.511 (0.032)
2b	1000	0	0.505 (0.029)	0.507 (0.029)	0.503 (0.028)	0.504 (0.029)
2b	1000	0.5	0.504 (0.058)	0.503 (0.031)	0.499 (0.031)	0.501 (0.031)
3	100	0	0.656 (0.095)	1.35×10^{14} (3.01×10^{15}) ^b	0.540 (0.040)	0.633 (0.082)
3	100	0.5	0.643 (0.093)	7.92×10^8 (1.62×10^{10}) ^b	0.542 (0.046)	0.626 (0.071)
3	500	0	0.525 (0.030)	0.568 (0.039)	0.507 (0.027)	0.518 (0.029)
3	500	0.5	0.522 (0.030)	0.568 (0.038)	0.506 (0.028)	0.517 (0.029)
3	1000	0	0.509 (0.026)	0.525 (0.028)	0.502 (0.026)	0.508 (0.026)
3	1000	0.5	0.508 (0.028)	0.524 (0.031)	0.501 (0.028)	0.506 (0.028)

Table A5: Variable-selection performance TPR μ , by DGP, n and ρ_X .

dgp	n	rho_x	cyclic	non-cyclic
1	100	0	1.000 (0.000)	1.000 (0.000)
1	100	0.5	1.000 (0.000)	1.000 (0.000)
1	500	0	1.000 (0.000)	1.000 (0.000)
1	500	0.5	1.000 (0.000)	1.000 (0.000)
1	1000	0	1.000 (0.000)	1.000 (0.000)
1	1000	0.5	1.000 (0.000)	1.000 (0.000)
2a	100	0	1.000 (0.000)	1.000 (0.000)
2a	100	0.5	1.000 (0.000)	1.000 (0.000)
2a	500	0	1.000 (0.000)	1.000 (0.000)
2a	500	0.5	1.000 (0.000)	1.000 (0.000)
2a	1000	0	1.000 (0.000)	1.000 (0.000)
2a	1000	0.5	1.000 (0.000)	1.000 (0.000)
2b	100	0	0.902 (0.298)	0.940 (0.238)
2b	100	0.5	0.958 (0.201)	0.936 (0.245)
2b	500	0	1.000 (0.000)	1.000 (0.000)
2b	500	0.5	1.000 (0.000)	1.000 (0.000)
2b	1000	0	1.000 (0.000)	1.000 (0.000)
2b	1000	0.5	1.000 (0.000)	1.000 (0.000)
3	100	0	1.000 (0.000)	1.000 (0.000)
3	100	0.5	1.000 (0.000)	1.000 (0.000)
3	500	0	1.000 (0.000)	1.000 (0.000)
3	500	0.5	1.000 (0.000)	1.000 (0.000)
3	1000	0	1.000 (0.000)	1.000 (0.000)
3	1000	0.5	1.000 (0.000)	1.000 (0.000)

Table A6: Variable-selection performance TPR σ , by DGP, n and ρ_X .

dgp	n	rho_x	cyclic	non-cyclic
1	100	0	0.999 (0.022)	0.998 (0.032)
1	100	0.5	0.995 (0.050)	0.996 (0.045)
1	500	0	1.000 (0.000)	1.000 (0.000)
1	500	0.5	1.000 (0.000)	1.000 (0.000)
1	1000	0	1.000 (0.000)	1.000 (0.000)
1	1000	0.5	1.000 (0.000)	1.000 (0.000)
2a	100	0	0.990 (0.100)	0.942 (0.234)
2a	100	0.5	0.966 (0.181)	0.898 (0.303)
2a	500	0	1.000 (0.000)	1.000 (0.000)
2a	500	0.5	0.998 (0.045)	0.992 (0.089)
2a	1000	0	1.000 (0.000)	1.000 (0.000)
2a	1000	0.5	1.000 (0.000)	0.998 (0.045)
2b	100	0	0.999 (0.022)	0.998 (0.032)
2b	100	0.5	0.999 (0.022)	0.999 (0.022)
2b	500	0	1.000 (0.000)	1.000 (0.000)
2b	500	0.5	1.000 (0.000)	1.000 (0.000)
2b	1000	0	1.000 (0.000)	1.000 (0.000)
2b	1000	0.5	1.000 (0.000)	1.000 (0.000)
3	100	0	0.921 (0.211)	0.970 (0.119)
3	100	0.5	0.918 (0.201)	0.933 (0.176)
3	500	0	1.000 (0.000)	1.000 (0.000)
3	500	0.5	1.000 (0.000)	1.000 (0.000)
3	1000	0	1.000 (0.000)	1.000 (0.000)
3	1000	0.5	1.000 (0.000)	1.000 (0.000)

Table A7: Variable-selection performance FPR μ , by DGP, n and ρ_X .

dgp	n	rho_x	cyclic	non-cyclic
1	100	0	0.620 (0.344)	0.364 (0.325)
1	100	0.5	0.648 (0.319)	0.394 (0.307)
1	500	0	0.712 (0.260)	0.434 (0.319)
1	500	0.5	0.799 (0.222)	0.547 (0.298)
1	1000	0	0.680 (0.251)	0.458 (0.321)
1	1000	0.5	0.764 (0.227)	0.539 (0.302)
2a	100	0	0.615 (0.353)	0.352 (0.324)
2a	100	0.5	0.616 (0.323)	0.372 (0.298)
2a	500	0	0.608 (0.348)	0.464 (0.330)
2a	500	0.5	0.553 (0.348)	0.431 (0.317)
2a	1000	0	0.586 (0.351)	0.442 (0.334)
2a	1000	0.5	0.537 (0.343)	0.427 (0.310)
2b	100	0	0.430 (0.299)	0.322 (0.267)
2b	100	0.5	0.541 (0.299)	0.318 (0.247)
2b	500	0	0.416 (0.367)	0.386 (0.299)
2b	500	0.5	0.517 (0.328)	0.462 (0.280)
2b	1000	0	0.462 (0.409)	0.379 (0.282)
2b	1000	0.5	0.513 (0.373)	0.494 (0.262)
3	100	0	0.314 (0.204)	0.084 (0.086)
3	100	0.5	0.336 (0.188)	0.136 (0.089)
3	500	0	0.330 (0.156)	0.148 (0.114)
3	500	0.5	0.375 (0.146)	0.215 (0.116)
3	1000	0	0.347 (0.114)	0.154 (0.120)
3	1000	0.5	0.392 (0.114)	0.228 (0.117)

Table A8: Variable-selection performance FPR σ , by DGP, n and ρ_X .

dgp	n	rho_x	cyclic	non-cyclic
1	100	0	0.750 (0.247)	0.577 (0.286)
1	100	0.5	0.775 (0.234)	0.570 (0.292)
1	500	0	0.904 (0.174)	0.527 (0.309)
1	500	0.5	0.893 (0.193)	0.547 (0.309)
1	1000	0	0.883 (0.165)	0.541 (0.305)
1	1000	0.5	0.914 (0.158)	0.499 (0.308)
2a	100	0	0.982 (0.067)	0.875 (0.154)
2a	100	0.5	0.960 (0.099)	0.842 (0.169)
2a	500	0	0.982 (0.067)	0.875 (0.161)
2a	500	0.5	0.962 (0.094)	0.857 (0.162)
2a	1000	0	0.977 (0.079)	0.849 (0.171)
2a	1000	0.5	0.960 (0.097)	0.817 (0.184)
2b	100	0	0.435 (0.384)	0.369 (0.292)
2b	100	0.5	0.393 (0.329)	0.383 (0.294)
2b	500	0	0.709 (0.297)	0.417 (0.304)
2b	500	0.5	0.654 (0.360)	0.449 (0.289)
2b	1000	0	0.739 (0.214)	0.425 (0.314)
2b	1000	0.5	0.789 (0.237)	0.435 (0.304)
3	100	0	0.397 (0.119)	0.293 (0.096)
3	100	0.5	0.370 (0.105)	0.257 (0.102)
3	500	0	0.484 (0.185)	0.244 (0.117)
3	500	0.5	0.442 (0.163)	0.260 (0.111)
3	1000	0	0.545 (0.088)	0.244 (0.125)
3	1000	0.5	0.499 (0.096)	0.251 (0.115)

Simulation Study Table DGP 4

Table A9: Performance of linear and spline base-learners in DGP 4. RMSE values are Monte Carlo means with Monte Carlo standard errors in parentheses. NLL values are medians with interquartile ranges in parentheses.

n	ρ_X	Learner	RMSE $_{\mu}$	RMSE $_{\eta_{\sigma}}$	NLL
100	0.0	Linear	2.010 (0.003)	0.824 (0.004)	1.338 (1.311–1.366)
		Spline	2.007 (0.017)	0.925 (0.018)	1.342 (1.256–102.431)
100	0.5	Linear	2.015 (0.003)	0.824 (0.004)	1.337 (1.312–1.372)
		Spline	2.006 (0.022)	0.933 (0.019)	1.361 (1.258–551.048)
500	0.0	Linear	2.001 (0.003)	0.836 (0.002)	1.317 (1.295–1.341)
		Spline	1.419 (0.015)	0.516 (0.007)	0.942 (0.778–1.192)
500	0.5	Linear	1.999 (0.003)	0.838 (0.002)	1.315 (1.296–1.339)
		Spline	1.363 (0.016)	0.489 (0.007)	0.878 (0.749–1.237)
1000	0.0	Linear	2.002 (0.003)	0.838 (0.001)	1.318 (1.295–1.342)
		Spline	1.145 (0.016)	0.399 (0.007)	0.698 (0.647–0.867)
1000	0.5	Linear	1.996 (0.003)	0.838 (0.001)	1.314 (1.296–1.336)
		Spline	1.093 (0.016)	0.376 (0.006)	0.681 (0.633–0.793)

Simulation Study Figures

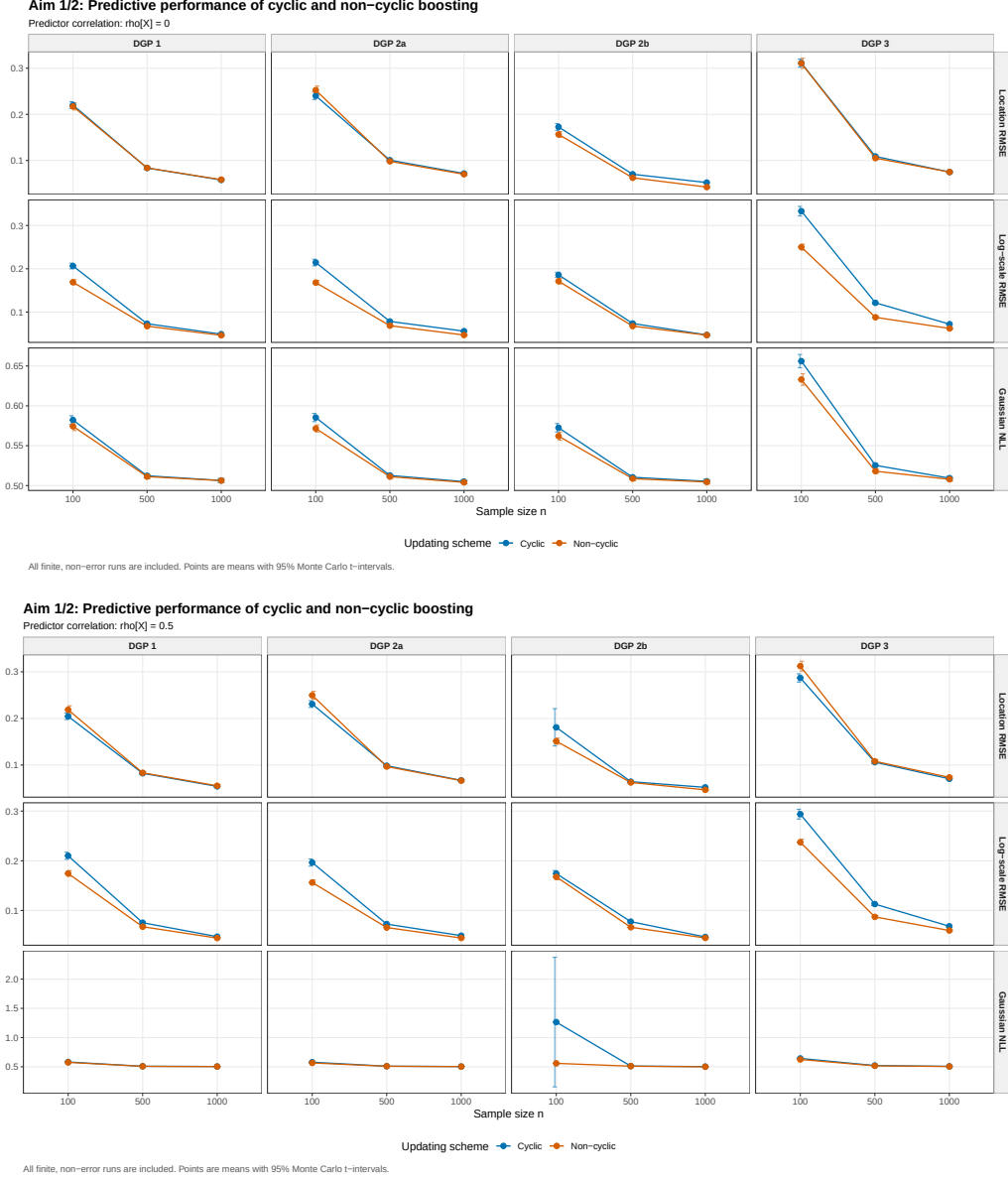


Figure A3: Predictive performance of cyclic and non-cyclic boosting with linear base-learners for $\rho_X = 0$ (top) and $\rho_X = 0.5$ (bottom). Points are Monte Carlo means and whiskers are 95% Monte Carlo t -intervals. Lower values indicate better performance. Each point is based on 500 replications, except for cyclic boosting in DGP 3 with $n = 100$ and $\rho_X = 0$, which is based on 498 non-error runs.

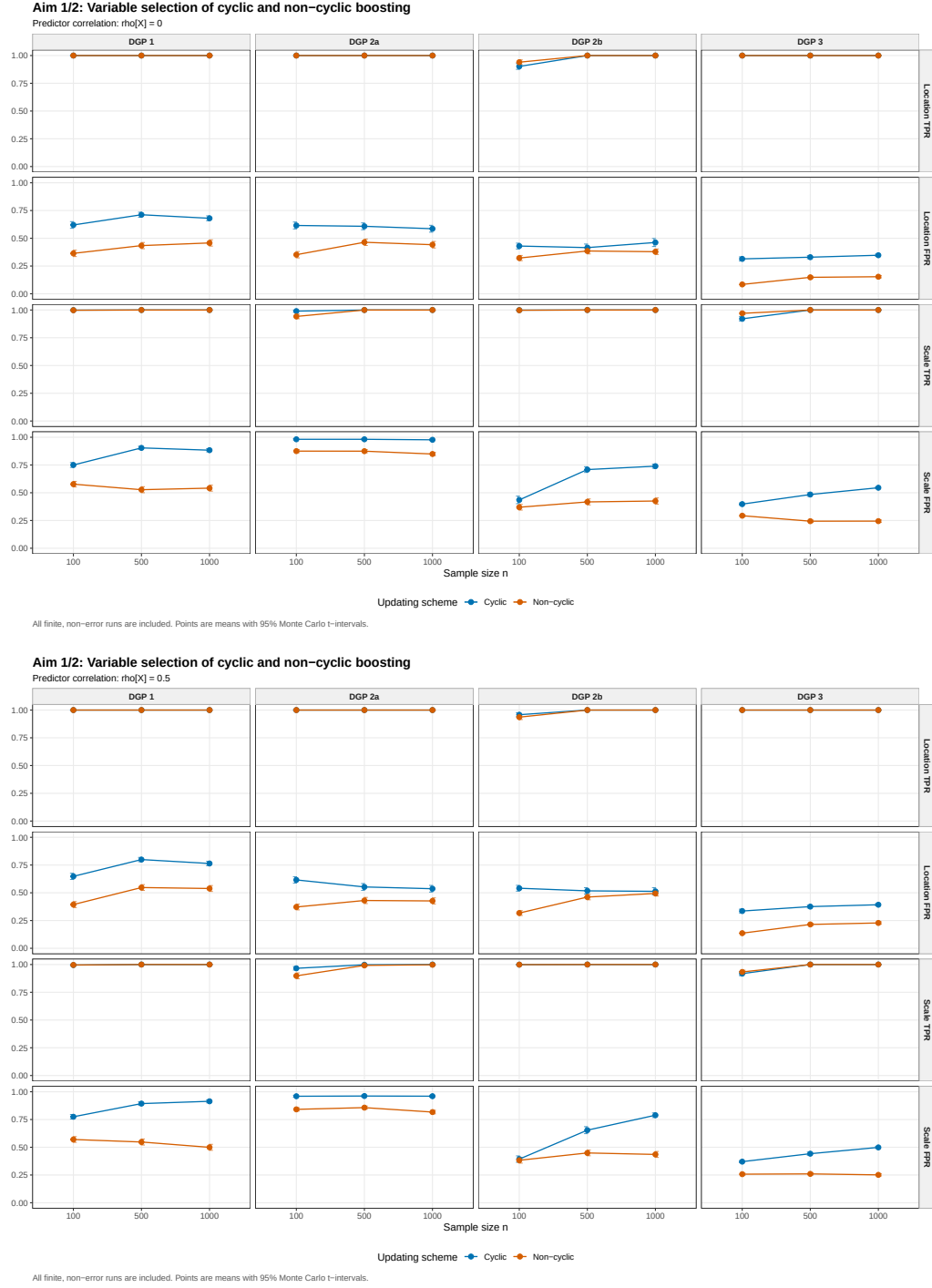
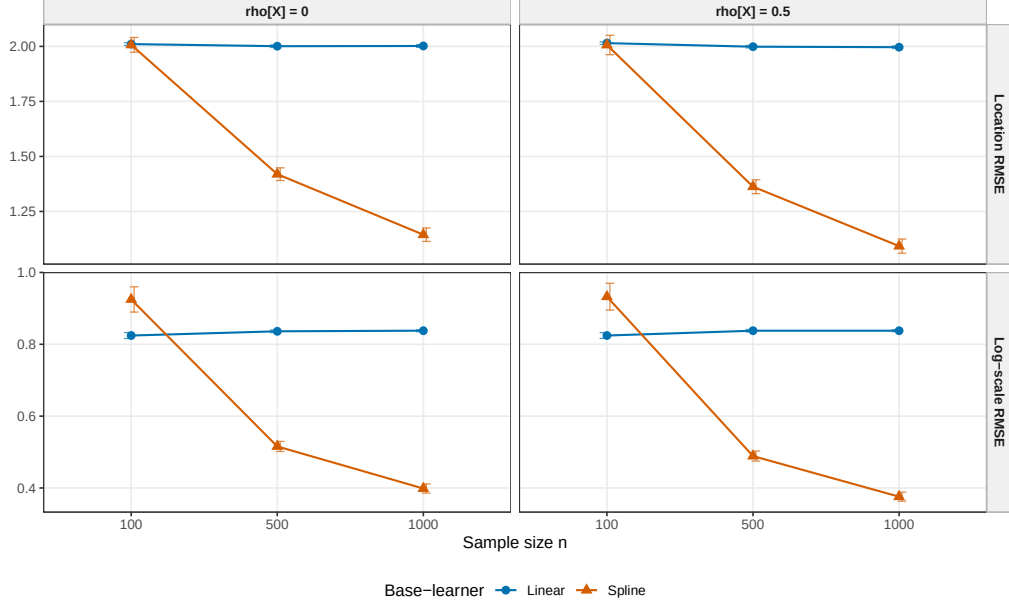


Figure A4: Variable-selection performance of cyclic and non-cyclic boosting with linear base-learners for $\rho_X = 0$ (top) and $\rho_X = 0.5$ (bottom). Points are Monte Carlo means and whiskers are 95% Monte Carlo t -intervals. High TPR and low FPR are desirable.

Aim 3: RMSE of linear and spline base-learners

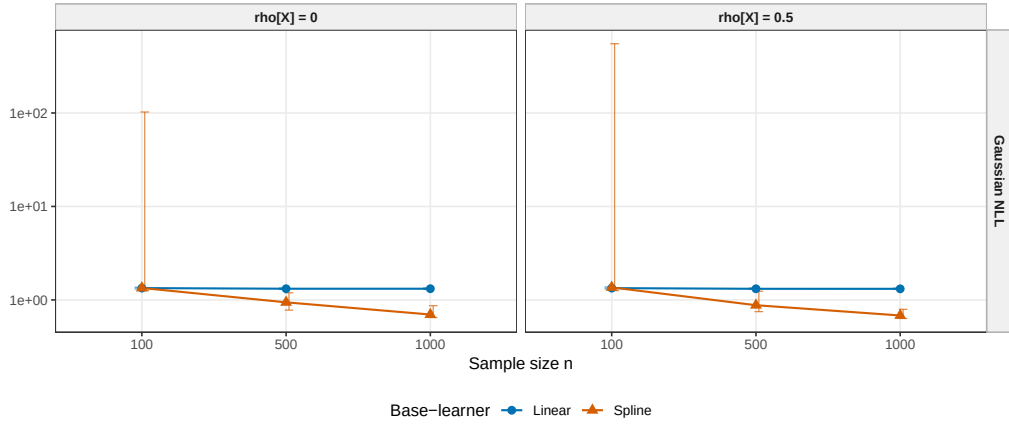
DGP 4: non-cyclic boosting



All finite, non-error runs are included, including warning runs. Points are means with 95% Monte Carlo t -intervals.

Aim 3: Typical Gaussian NLL

DGP 4: non-cyclic boosting



All finite, non-error runs are included, including warning runs. Points are medians and whiskers are interquartile ranges; y-axis is logarithmic.

Figure A5: Performance of non-cyclic linear and spline base-learners in the nonlinear DGP 4. Top: mean location and log-scale RMSE with 95% Monte Carlo t -intervals. Bottom: median Gaussian NLL with the IQR on a logarithmic axis. The median is used for NLL because rare extreme spline predictions make the arithmetic mean unrepresentative.

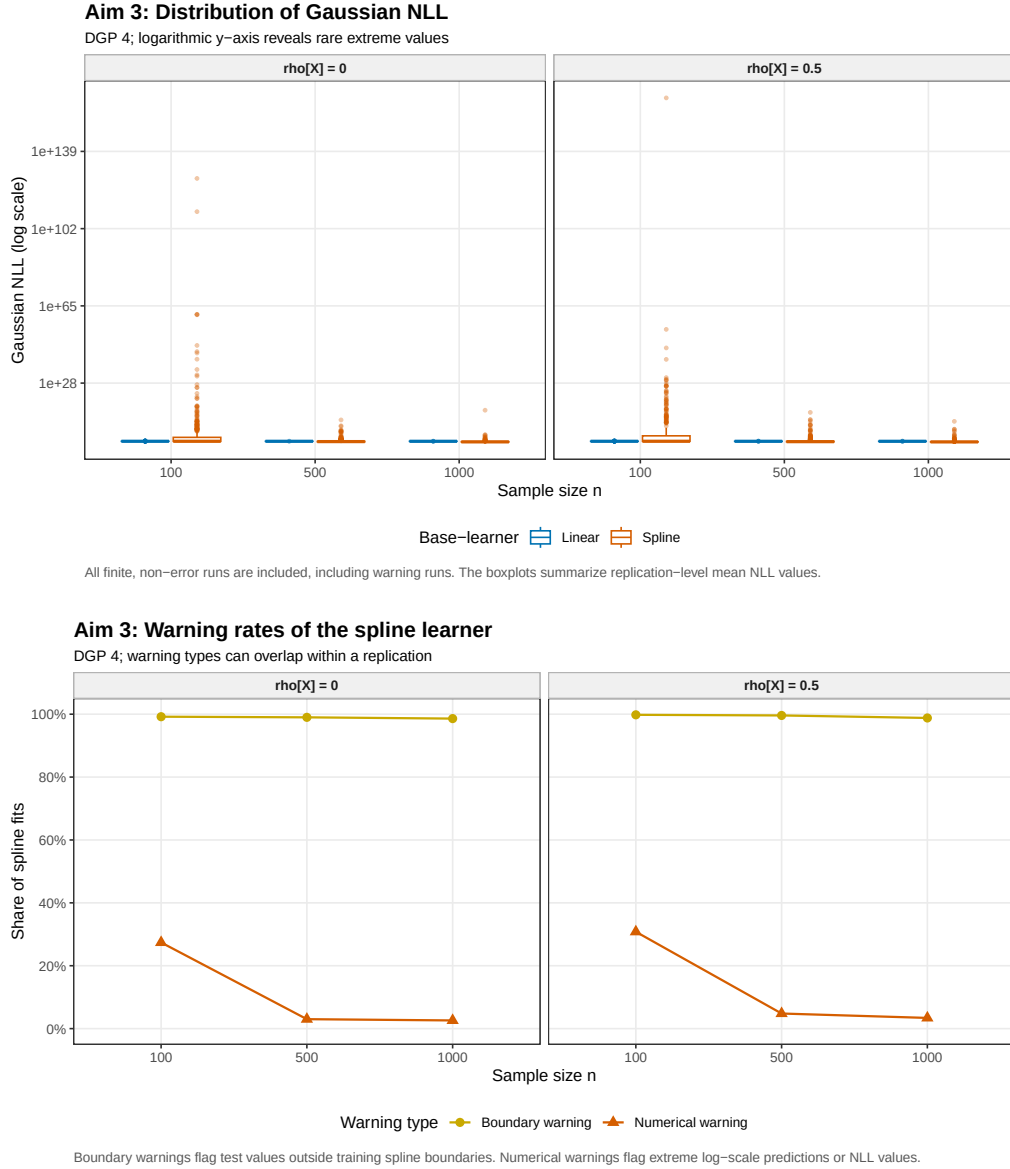


Figure A6: Distribution and diagnostics of the NLL in DGP 4. Top: replication-level Gaussian NLL on a logarithmic scale. Bottom: diagnostic-warning rates for the spline base-learner. Boundary and numerical warnings can occur in the same replication. A boundary warning indicates test observations outside the training boundary knots; a numerical warning indicates extreme log-scale predictions or NLL contributions.

Coverage

lmls.boosting coverage - 100.00%

Files	Source						
File	Lines	Relevant	Covered	Missed	Hits / Line	Coverage	
R/boost_lmls.R	571	260	260	0	18	100.00%	
R/early_stop.R	441	204	204	0	12	100.00%	
R/tune_lmls.R	324	133	133	0	11	100.00%	
R/s3_predict.R	196	74	74	0	9	100.00%	
R/s3_summary.R	217	72	72	0	6	100.00%	
R/s3_plot.R	128	72	72	0	6	100.00%	
R/data_prep.R	182	64	64	0	34	100.00%	
R/weak_updates.R	247	62	62	0	21	100.00%	
R/weak_learner_spline.R	153	58	58	0	7	100.00%	
R/predict_spline_path.R	88	33	33	0	7	100.00%	
R/weak_learner_linear.R	67	27	27	0	75	100.00%	
R/best_learner.R	54	15	15	0	41	100.00%	
R/initialization.R	57	14	14	0	23	100.00%	
R/gradients.R	46	2	2	0	32	100.00%	
R/loglik.R	32	2	2	0	11	100.00%	

Figure A7: Code coverage report for the R package generated using `covr`, showing the `fig:plot4` of the source files and the number of test executions (hits) for individual lines of code.

A.2 Reproducibility

The code for data generation, model implementation, simulation execution, and analysis, together with the generated simulation data, is available at "<https://owncloud.gwdg.de/index.php/s/o8m0tfQr37kb0Jc>" (password: "asp26boosting"). The simulation and analysis scripts require R version 4.1 or higher, as they make regular use of the native pipe operator (`|>`). The simulation study materials are organized into three main directories, described below. A README file in the root directory provides detailed documentation of the simulation design, computational workflow, execution of the simulation runs, output files, and reproducibility checks:

- **scripts/**: Contains the R scripts for running the main simulation study, the nonlinear spline study, the optional GAMLSS benchmark, and for generating the figures.
- **results/**: Contains the outputs of the simulation runs, including the final results of the main study, the spline study, and the optional GAMLSS benchmark, as well as pilot and archived results.

- `figures/`: Contains the figures generated from the final simulation results.

A.3 Package Manual

The following pages contain the package manual automatically generated from the R package documentation.

Package ‘lmls.boosting’

September 14, 2026

Title Gradient Boosting for Location-Scale Models

Version 0.0.0.9000

Description Description: Implements component-wise gradient boosting for Gaussian location-scale models. Separate additive predictors are used for the conditional mean and the logarithm of the conditional standard deviation. Linear or spline base learners are fitted iteratively using functional gradient descent. Early stopping enables regularization and variable selection. The package provides functions for model fitting, cross-validation, hyperparameter tuning, prediction, and visualization.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Suggests knitr,
rmarkdown,
testthat (>= 3.0.0)

Config/testthat/edition 3

Config/roxygen2/version 8.0.0

RoxygenNote 8.0.0

VignetteBuilder knitr

Contents

lmls.boosting-package	2
boost_lmls	2
plot.boost_lmls	4
predict.boost_lmls	5
print.summary.boost_lmls	6
summary.boost_lmls	7
tune_lmls	8

Index	11
--------------	-----------

lmls.boosting-package *lmls.boosting*

Description

Description: Implements component-wise gradient boosting for Gaussian location-scale models. Separate additive predictors are used for the conditional mean and the logarithm of the conditional standard deviation. Linear or spline base learners are fitted iteratively using functional gradient descent. Early stopping enables regularization and variable selection. The package provides functions for model fitting, cross-validation, hyperparameter tuning, prediction, and visualization.

Author(s)

Maintainer: Rudi Murat Meyer <rudi.meyer@stud.uni-goettingen.de>

Authors:

- Rudi Murat Meyer <rudi.meyer@stud.uni-goettingen.de>
- Ferran Vila <ferran.vila@stud.uni-goettingen.de>
- Sophie Eileen Wachtel <sophie.wachtel@stud.uni-goettingen.de>
- Ben Lenard Wais <benlenard.wais@stud.uni-goettingen.de>

boost_lmls

Boosting for Location-Scale Models

Description

Component-wise gradient boosting for simultaneous estimation of the location and scale parameters of a Gaussian regression model.

Usage

```
boost_lmls(
  formula_mu,
  formula_sigma = formula_mu,
  data,
  mstop,
  nu_mu = 0.1,
  nu_sigma = 0.1,
  method = c("non-cyclic", "cyclic"),
  learner = c("linear", "spline"),
  spline_degree = 3,
  spline_nknots = 5,
  spline_knot_strategy = c("quantile", "equidistant"),
  cv = FALSE,
  k = 5,
  progress = FALSE
)
```

Arguments

formula_mu	A model formula specifying the predictors for the location parameter μ .
formula_sigma	A model formula specifying the predictors for the log-scale parameter $\log(\sigma)$. Defaults to formula_mu.
data	A data frame containing the response and predictor variables.
mstop	Positive integer specifying the stopping iteration(s) when <code>cv = FALSE</code> , and the maximum candidate stopping iteration(s) considered during cross-validation when <code>cv = TRUE</code> . For <code>method = "cyclic"</code> , either a single value or a length-two vector <code>c(mstop_mu, mstop_sigma)</code> may be supplied. For <code>method = "non-cyclic"</code> , exactly one value must be supplied.
nu_mu	Learning rate for updates of the location component. Must lie in $(0, 1]$.
nu_sigma	Learning rate for updates of the scale component. Must lie in $(0, 1]$.
method	Boosting update strategy. Either <code>"cyclic"</code> or <code>"non-cyclic"</code> .
learner	Base learner type. Either <code>"linear"</code> or <code>"spline"</code> .
spline_degree	Degree of the B-spline basis. Used only when <code>learner = "spline"</code> .
spline_nknots	Number of internal spline knots. Used only when <code>learner = "spline"</code> .
spline_knot_strategy	Knot placement strategy. <code>"quantile"</code> places internal knots at empirical quantiles of the predictor, whereas <code>"equidistant"</code> places them at equally spaced points over the observed predictor range. Used only when <code>learner = "spline"</code> .
cv	Logical indicating whether the stopping iteration(s) should be selected using K-fold cross-validation.
k	Number of cross-validation folds. Used only when <code>cv = TRUE</code> .
progress	Logical. If <code>TRUE</code> , information about selected base learners is printed during fitting.

Details

The model assumes

$$Y_i \sim N(\mu_i, \sigma_i^2),$$

where the location and scale predictors are modeled as

$$\mu_i = f_\mu(\mathbf{x}_i)$$

and

$$\log(\sigma_i) = f_\sigma(\mathbf{x}_i).$$

Both predictors are estimated by component-wise gradient boosting. At each boosting iteration, candidate base learners are fitted to the corresponding negative gradient and the best-performing learner is selected.

With `method = "cyclic"`, the location and scale components are updated sequentially. Separate stopping iterations may be specified for the two components.

With `method = "non-cyclic"`, candidate updates for both parameters are compared at each iteration and only the update yielding the smaller loss is applied.

If `cv = TRUE`, cross-validation is used to determine the stopping iteration(s) before the final model is fitted.

Value

An object of class `boost_lmls`, containing:

beta0, beta Intercept and coefficients of the location component for linear base learners.
gamma0, gamma Intercept and coefficients of the scale component for linear base learners.
mu Fitted location values.
sigma Fitted conditional standard deviations.
y Observed response vector.
mstop Stopping iteration(s) used for fitting.
nu_mu, nu_sigma Learning rates.
selection_mu, selection_sigma Relative base learner selection frequencies for the two model components.
betapath, gammapath Coefficient paths for linear base learners.
weak_learners_mu, weak_learners_sigma Selected base learners along the boosting path.
X_mu, Z_sigma Design matrices used for fitting.
method Boosting update strategy.
learner Base learner type.
call Matched model call.

Examples

```
set.seed(123)
n <- 100
x1 <- runif(n, -1, 1)
x2 <- runif(n, -1, 1)
mu <- 1 + 2 * x1
sigma <- exp(0.5 + 0.5 * x2)
y <- rnorm(n, mean = mu, sd = sigma)
dat <- data.frame(y, x1, x2)

mod <- boost_lmls(y ~ x1, formula_sigma = ~ x2, data = dat,
                  mstop = 20, learner = "linear")
print(mod)
```

plot.boost_lmls	<i>Plot a boost_lmls model</i>
-----------------	--------------------------------

Description

S3 method for class `boost_lmls`. For linear base learners, shows the coefficient values in relation to the number of iterations for both location (`mu`) and scale (`sigma`). Additionally, shows the base learner selection frequencies as a barplot, either separately for `mu/sigma` (cyclic method) or combined and color-coded (non-cyclic method).

Usage

```
## S3 method for class 'boost_lmls'
plot(x, ...)
```

Arguments

`x` An object of class `boost_lmls`.
`...` Additional arguments passed to the plot method.

Value

Invisibly returns `NULL`.

Examples

```
set.seed(123)
n <- 200
x1 <- runif(n, -1, 1)
x2 <- runif(n, -1, 1)
x3 <- runif(n, -1, 1)
mu <- 2 + 1.5 * x1
sigma <- exp(0.5 + 0.7 * x3)
y <- rnorm(n, mean = mu, sd = sigma)
dat <- data.frame(y, x1, x2, x3)

mod <- boost_lmls(y ~ x1 + x2 + x3, mstop= 100, formula_sigma = ~ x1 + x2 + x3,
                  data = dat, learner = "linear")

plot(mod)
```

`predict.boost_lmls` *Predict method for boost_lmls objects*

Description

Obtains predictions for either the location (`mu`) or scale (`sigma`) parameter of a fitted `boost_lmls` model, either on the original training data or for new observations.

Usage

```
## S3 method for class 'boost_lmls'
predict(object, newdata = NULL, parameter = "mu", type = "response", ...)
```

Arguments

`object` A fitted model object of class `boost_lmls`.
`newdata` Optional data frame containing the predictor variables to predict for. Must contain the same predictors used to fit the requested parameter. If `NULL` (default), predictions are returned for the original training data.
`parameter` Character string specifying which distribution parameter to predict. Either `"mu"` (default) or `"sigma"`.
`type` Character string specifying the type of prediction. Either `"response"` (default), which returns the predicted values on the response scale, or `"terms"`, which returns the per-predictor contributions and the intercept separately.
`...` Additional arguments, currently unused.

Value

If type = "response", a numeric vector of predicted values. If type = "terms", a list with elements coefficients (a matrix of per-predictor contributions) and intercept.

Examples

```
## Not run:
set.seed(161)
n <- 200
x1 <- rnorm(n)
x2 <- rnorm(n)
mu <- 2 + 1.5 * x1 - 0.5 * x2
sigma <- exp(0.3 + 0.2 * x1)
y <- rnorm(n, mean = mu, sd = sigma)
sim_data <- data.frame(y = y, x1 = x1, x2 = x2)

model <- boost_lmls(
  formula_mu = y ~ x1 + x2,
  data = sim_data,
  mstop = 100
)

predict(model, type = "response")
predict(model, newdata = sim_data, parameter = "sigma", type = "terms")

## End(Not run)
```

```
print.summary.boost_lmls
```

Print method for summary.boost_lmls objects

Description

Displays a formatted summary of a fitted boost_lmls model: the matched call, the assumed distribution and loss functions for mu and sigma, the number of boosting iterations, learning rates, base learner selection frequencies, and learner-specific output (coefficients for linear learners, selection details for splines).

Usage

```
## S3 method for class 'summary.boost_lmls'
print(x, ...)
```

Arguments

x	An object of class summary.boost_lmls.
...	Additional arguments passed to the method.

Value

Invisibly returns x

Examples

```
set.seed(161)
n <- 200
x1 <- runif(n, -1, 1)
x2 <- runif(n, -1, 1)
x3 <- runif(n, -1, 1)
mu <- 2 + 1.5 * x1
sigma <- exp(0.5 + 0.7 * x3)
y <- rnorm(n, mean = mu, sd = sigma)
dat <- data.frame(y, x1, x2, x3)

mod <- boost_lmls(y ~ x1 + x2 + x3, formula_sigma = ~ x1 + x2 + x3, mstop=100,
                  data = dat, learner = "linear")
print(summary(mod))
```

summary.boost_lmls	<i>Summary method for a boost_lmls object</i>
--------------------	---

Description

Produces a summary of a fitted `boost_lmls` model, including the number of boosting iterations, learning rates, base learner selection frequencies for the location (`mu`) and scale (`sigma`) submodels, and — for linear base learners — the estimated coefficients.

Usage

```
## S3 method for class 'boost_lmls'
summary(object, ...)
```

Arguments

<code>object</code>	An object of class <code>boost_lmls</code> .
<code>...</code>	Additional arguments passed to the method.

Value

An object of class `summary.boost_lmls`, containing:

- mstop** Stopping iteration(s) used for fitting.
- nu_mu, nu_sigma** Learning rates for the location and scale components.
- selection_mu, selection_sigma** Relative selection frequencies, sorted in decreasing order.
- coefficients_mu, coefficients_sigma** Estimated coefficients for linear base learners.
- r2_coxsnell** Cox–Snell pseudo- R^2 .
- r2_nagelkerke** Nagelkerke pseudo- R^2 , if defined.
- call** Matched model call.
- learner** Base learner type.
- method** Boosting update strategy.
- spline_degree, spline_nknots, spline_strategy** Spline configuration, if applicable.

Examples

```

set.seed(161)
n <- 200
x1 <- runif(n, -1, 1)
x2 <- runif(n, -1, 1)
x3 <- runif(n, -1, 1)
mu <- 2 + 1.5 * x1
sigma <- exp(0.5 + 0.7 * x3)
y <- rnorm(n, mean = mu, sd = sigma)
dat <- data.frame(y, x1, x2, x3)

mod <- boost_lmls(y ~ x1 + x2 + x3, formula_sigma = ~ x1 + x2 + x3, mstop=100,
                  data = dat, learner = "linear")

summary(mod)

```

tune_lmls

*Hyperparameter Tuning for Location-Scale Boosting***Description**

Tunes the learning rate and early-stopping iterations of a location-scale boosting model using K-fold cross-validation.

Usage

```

tune_lmls(
  formula_mu,
  formula_sigma = formula_mu,
  data,
  k = 5,
  mstop_max = 200,
  nu_min,
  nu_max,
  n_nu = 5,
  method = c("non-cyclic", "cyclic"),
  learner = c("linear", "spline"),
  spline_degree = 3,
  spline_nknots = 5,
  spline_knot_strategy = c("quantile", "equidistant")
)

```

Arguments

formula_mu	Formula for the location model.
formula_sigma	Formula for the log-standard-deviation model.
data	Data frame containing the variables.
k	Integer. Number of CV folds.
mstop_max	Maximum number of boosting iterations. For cyclic boosting, may be a vector of length two: c(max_mu, max_sigma).

<code>nu_min</code>	Smallest learning rate considered.
<code>nu_max</code>	Largest learning rate considered.
<code>n_nu</code>	Integer. Number of learning rates in the tuning grid.
<code>method</code>	Character. Boosting update strategy. Either "non-cyclic" or "cyclic".
<code>learner</code>	Base learner type. Either "linear" or "spline".
<code>spline_degree</code>	Degree of the B-spline basis when learner = "spline".
<code>spline_nknots</code>	Number of internal spline knots. Used only when learner = "spline".
<code>spline_knot_strategy</code>	Knot placement strategy. "quantile" places internal knots at empirical quantiles of the predictor, whereas "equidistant" places them at equally spaced points over the observed predictor range. Used only when learner = "spline".

Details

A common learning rate is used for the location and scale components: $\nu_\mu = \nu_\sigma = \nu$. For every candidate learning rate, early stopping is performed using the same CV folds. The combination with the smallest mean validation loss is selected.

Value

A list containing:

best_nu Selected learning rate.
best_mstop_mu Selected stopping iteration for mu (cyclic only).
best_mstop_sigma Selected stopping iteration for sigma (cyclic only).
best_mstop Selected total stopping iteration (non-cyclic only).
best_cv_loss Smallest mean CV loss.
results Data frame containing all tuning results.
folds Fold assignment used for tuning.

Examples

```
set.seed(123)
n <- 100
x1 <- runif(n, -1, 1)
x2 <- runif(n, -1, 1)
mu <- 1 + 2 * x1
sigma <- exp(0.5 + 0.5 * x2)
y <- rnorm(n, mean = mu, sd = sigma)
dat <- data.frame(y, x1, x2)

## Not run:
tuned <- tune_lmls(
  formula_mu = y ~ x1,
  formula_sigma = ~ x2,
  data = dat,
  k = 3,
  mstop_max = 30,
  nu_min = 0.05,
  nu_max = 0.2,
  n_nu = 3,
```

```
    learner = "linear"  
  )  
  print(tuned)  
  
## End(Not run)
```

Index

`boost_lmls`, [2](#)

`lmls.boosting (lmls.boosting-package)`, [2](#)

`lmls.boosting-package`, [2](#)

`plot.boost_lmls`, [4](#)

`predict.boost_lmls`, [5](#)

`print.summary.boost_lmls`, [6](#)

`summary.boost_lmls`, [7](#)

`tune_lmls`, [8](#)

Declaration

I hereby declare that I have produced this work independently and without outside assistance, and have used only the sources and tools stated.

I have clearly identified the sources of any sections from other works that I have quoted or given in essence.

I have complied with the guidelines on good academic practice at the University of Göttingen.

I am aware that failure to comply with these principles will result in the examination being graded “nicht bestanden”, i.e. failed.

In the present work, we have used ChatGPT or another AI as follows:

- ☐ Not at all
- ☐ For brainstorming ideas
- ☐ For developing the outline
- ☐ For generating individual passages, accounting for a total of ... % of the entire text
- ☒ For developing software source code
- ☒ For optimizing or restructuring software source code
- ☒ For proofreading or optimizing text
- ☐ Other, specify: ...

September 15, 2026



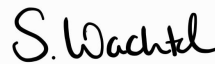
Meyer, Rudi Murat

September 15, 2026



Vila, Ferran

September 15, 2026



Wachtel, Sophie Eileen

September 15, 2026



Wais, Ben Lenard